ANÁLISIS TÉCNICO / 01

# Informe de análisis del backend

FACTURACIÓN E INVENTARIO / PROYECTO NATHALY

## Evaluación y ajuste frente a la arquitectura propuesta en *Diseño de Sistemas de Internet*.

Fecha de revisión: 11 de octubre de 2026. Entorno: repositorios locales, SQL Server local y GitHub. Enfoque: tecnología backend, patrón de diseño, estrategias de desarrollo y evidencia verificable.

| Resultado | Conclusión |
| --- | --- |
| Arquitectura | La base cliente-servidor y las tres capas lógicas ya existían. Se incorporó una fachada transaccional para la confirmación directa de facturas. |
| Ajustes ejecutados | Controlador reducido, validación de datos, bloqueos de stock, idempotencia persistente, autorización en API, revocación de sesiones y TypeScript en el flujo de facturación del cliente. |
| Evidencia | 14 pruebas unitarias y 4 de integración local aprobadas. Frontend: comprobación de tipos, build y lint sin errores; quedan 15 advertencias. |
| Alcance del dictamen | Alineación del núcleo arquitectónico. No se certifica cumplimiento total de todos los requisitos ni operación en producción: rendimiento, restauración, HTTPS y QA integral requieren evidencia adicional. |

## Por qué este análisis aporta al proyecto

La API concentra decisiones comerciales y protege la consistencia entre factura e inventario. Formalizar su diseño permite a distintos desarrolladores modificar reglas, comprobar fallos y revisar cambios sin repartir la coordinación entre pantallas o llamadas SQL inconexas.

El documento base identifica Tiendas Esther; los repositorios y la interfaz identifican Nathaly. Se conserva el nombre existente y se hace explícita la diferencia documental, sin asumir que ambas organizaciones son el mismo cliente.

ANÁLISIS TÉCNICO / 02

# Método y correspondencia arquitectónica

Se revisaron las 59 páginas del PDF, su texto y las tablas/diagramas relevantes renderizados, los dos repositorios, configuraciones, rutas, entidades, scripts SQL y pruebas. Las páginas citadas a continuación son físicas: el capítulo II ocupa 53-59, equivalentes a las impresas 52-58.

Tres capas lógicas; el cliente no tiene credenciales ni acceso directo a SQL Server.

| Propuesta del documento | Evidencia / evaluación posterior |
|---|---|
| Cliente-servidor, págs. 53-55 | React consume /api/v1 mediante Axios; Express procesa operaciones y se conecta a SQL Server. Alineado. |
| Tres niveles lógicos, págs. 53-54 | Presentación en front-nathaly; aplicación en api-nathaly; persistencia SQL. Capas lógicas no implican tres máquinas físicas. |
| React y TypeScript, pág. 53 | TypeScript añadido a entrada, servicio y formulario de facturación. El cliente conserva JSX/JS en otros módulos: adopción incremental, no migración total. |
| Facade, págs. 55-58 | FacturacionFacade.confirmar coordina validación, disponibilidad, registro y comprobante, dentro de la unidad de trabajo SQL. |
| Atomicidad, págs. 54 y 57 | Transacción exterior abarca factura/detalles/stock/Kardex, clave de reintento y reporte. Verificada en integración local. |
| Seguridad centralizada, pág. 55 | Autenticación y autorización antes de las rutas operativas. Permisos vigentes se consultan en servidor. |
| Evitar microservicios prematuros, págs. 58-59 | Se conserva una API central con módulos. No se distribuye la transacción de inventario. |

**Criterio de evidencia:** una ruta encontrada demuestra presencia de código; una prueba demuestra el caso ejercitado. Ninguna de ellas demuestra por sí sola disponibilidad, carga sostenida ni todas las reglas comerciales.

ANÁLISIS TÉCNICO / 03

# Tecnología backend y decisiones

## Node.js y Express 5.1.0

El backend utiliza Node.js y Express para exponer una API HTTP versionada. Mantener este stack aprovecha el código y el conocimiento existentes; su modelo asíncrono encaja con consultas SQL y solicitudes de red. Express permite componer autenticación, autorización, rutas y manejo de errores sin imponer una migración de framework.

Su flexibilidad exige disciplina: el framework no separa automáticamente negocio y persistencia. Por ello se introduce una fachada y contratos de colaboradores. Para producción deben configurarse TLS, tratamiento de entradas y cookies seguras, siguiendo la documentación de Express [2]. No se atribuye rendimiento real a la elección del framework sin medirlo.

## TypeScript 5.8.3

TypeScript describe solicitudes, detalles, resultados y contratos de la unidad de trabajo. Detecta incompatibilidades al compilar y facilita que cinco desarrolladores integren cambios con límites claros. El tipado no valida automáticamente los datos HTTP: la fachada también verifica números, cantidades, descuentos, productos repetidos y clave de idempotencia.

El backend ya utilizaba TypeScript. El cliente ahora usa TS/TSX en su entrada, servicio de ventas y formulario de factura; `allowJs` conserva componentes heredados. Esta transición sigue el enfoque de migración gradual de TypeScript [3]. La cobertura restante debe ampliarse por módulos, sin afirmar que todo el frontend esté tipado.

## API HTTP como frontera del sistema

La interfaz envía identificadores y detalles, pero no decide cómo escribir la factura ni descontar stock. El servidor devuelve respuestas de negocio y aplica permisos incluso ante llamadas directas. El contrato conserva los campos de respuesta usados por el cliente y añade `reutilizada` para distinguir un reintento.

| Beneficio en Nathaly | Costo o límite |
|---|---|
| Una frontera estable entre frontend y backend | Cambios de contrato requieren publicar ambos componentes compatibles. |
| Código modular y pruebas con colaboradores sustituibles | La separación se formaliza primero en facturación; otros controladores todavía requieren refactorización. |
| Sin cambio innecesario de plataforma | La versión exacta y sus dependencias deben mantenerse mediante lockfiles y revisión de vulnerabilidades. |

ANÁLISIS TÉCNICO / 04

# Persistencia, consistencia y seguridad

## SQL Server + TypeORM 0.3.31 + mssql 11.0.2

SQL Server conserva datos relacionales, restricciones, consecutivos y procedimientos existentes. Una base central evita replicar entre tiendas el estado de ventas y existencias. Las transacciones son la herramienta para confirmar una unidad lógica completa o revertirla [4]. Esta elección encaja con el alcance del documento; no implica que cualquier consulta sea correcta o rápida por utilizar SQL Server.

TypeORM aporta entidades y acceso a catálogos y seguridad. El controlador mssql permite procedimientos, parámetros y TVP para los detalles. La combinación responde a la base heredada: no se reescriben las reglas comerciales en otro ORM durante el ajuste. Su costo es mantener dos mecanismos de acceso y sus convenciones.

En el nuevo caso de uso, cada Request se crea con la misma Transaction de mssql [5]. No se mezcla una consulta TypeORM ajena a esa conexión dentro de la transacción de facturación. El procedimiento existente sigue escribiendo factura, detalles, stock y Kardex; la fachada agrega coordinación exterior, protección de reintentos y lectura del comprobante.

## Autenticación, autorización y sesiones

bcryptjs verifica contraseñas; JWT identifica una sesión de duración definida. La cookie es HttpOnly, SameSite Strict y Secure en producción. Se eliminó la devolución del token en JSON. Un identificador jti único distingue sesiones y una tabla de huellas revocadas permite rechazar el token tras cerrar sesión.

La política de permisos se aplica en la API y sólo admite asignaciones y permisos activos. Consultar un catálogo para facturar no concede autorización para editarlo. Los recursos agregados se restringen de forma conservadora a administración; esta matriz debe validarse con el responsable del negocio.

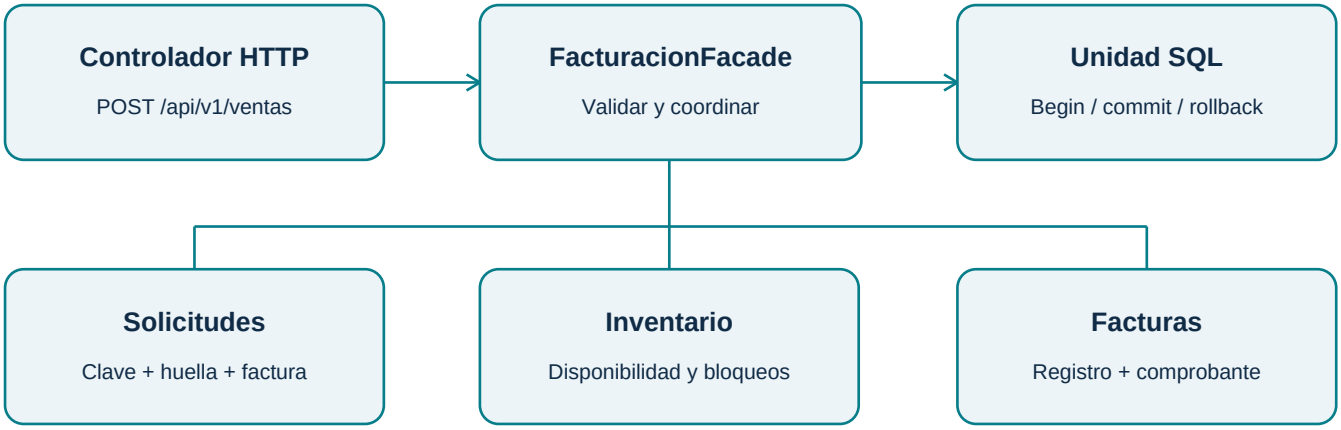
## Configuración y esquema versionados

Credenciales y clave JWT permanecen fuera del repositorio; .env.example documenta nombres sin secretos. Las migraciones 002-004 se ejecutan como tarea de despliegue, no durante HTTP. Se versionaron las definiciones revisadas de los procedimientos de registro y reporte. Todavía falta una línea base completa del esquema y de todos los procedimientos para reconstruir la base desde cero.

ANÁLISIS TÉCNICO / 05

# Patrón Facade aplicado a facturación

Facade ofrece una entrada simple para una operación que necesita varios colaboradores. En este proyecto, el controlador llama `confirmar` y adapta el resultado a HTTP; la fachada conserva la coordinación del caso de uso. El diagrama conceptual del documento se materializa con colaboradores definidos, sin crear una clase que concentre todos los módulos.




Los tres colaboradores usan la conexión de la misma transacción. Escritura SQL heredada encapsulada.

| Antes | Después |
|---|---|
| Validación, TVP, procedimiento y reporte dentro del controlador | Controlador delgado; validación y coordinación en fachada; detalles SQL en adaptador. |
| Validación de stock sin protección suficiente frente a rivales | UPDLOCK/HOLDLOCK conserva el bloqueo hasta terminar; productos ordenados y duplicados rechazados. |
| Un POST repetido podía generar otra venta | UUID por intento lógico, hash normalizado, registro persistente y bloqueo por usuario/clave. |
| Reporte leído después de la escritura confirmada | Reporte y registro de idempotencia dentro de la transacción exterior; falla provoca reversión. |
| Coordinación difícil de aislar en pruebas | Contratos TransaccionesFacturacion y UnidadFacturacion permiten verificar secuencia y abortos. |

## Regla transaccional y límite del patrón

Primera solicitud válida: 201. Reintento con la misma clave y contenido: 200 con la factura existente. Misma clave y contenido distinto: 409. La clave se conserva en el formulario ante un error de red; un bloqueo inmediato impide doble clic simultáneo.

Facade organiza; no sustituye las transacciones ni las restricciones. La escritura de inventario sigue encapsulada en el procedimiento heredado. Conversión de pedidos, anulación y devoluciones conservan otros flujos y necesitan pruebas y separación propias. Observer no se usa para el descuento obligatorio de stock porque ese paso debe formar parte de la misma unidad de trabajo.

ANÁLISIS TÉCNICO / 06

# Estrategias de desarrollo y Git Flow

## Trabajo por feature y evidencia de revisión

Se trabaja desde una rama feature creada a partir de develop. Los commits siguen Conventional Commits y agrupan responsabilidades: facturación, seguridad, dependencias y documentación. Un pull request explica el problema, el cambio, las migraciones y las pruebas; otra persona revisa antes de integrar.

| Paso | Responsabilidad y condición |
|---|---|
| feature/* -> develop | Desarrollador: actualizar develop, crear feature, implementar, ejecutar pruebas y subir su rama. PR con evidencia y revisión. |
| develop -> qa | Coordinador: seleccionar conjunto estable. QA aplica migraciones y verifica regresión, permisos, transacciones y criterios de cada feature. |
| qa -> master | Responsable de entrega: promover sólo lo aprobado por QA; registrar versión, respaldo y plan de reversión. |
| CI | Backend verifica compilación/pruebas y destino del PR. Frontend verifica lint y build, que incluye typecheck. Integración SQL es explícita contra QA, no usa la base privada desde CI. |

```
git clone https://github.com/cvargas0503/nathaly-backend.git
cd nathaly-backend
git switch develop
git pull --ff-only origin develop
git switch -c feature/mi-requerimiento
npm ci
npm test
git add <archivos-de-la-feature>
git commit -m "feat(modulo): describir el resultado"
git push -u origin feature/mi-requerimiento
```

Después se abre el PR con base develop, se atiende la revisión y se espera la aprobación. No se suben cambios directamente a develop, qa o master. La integración a qa y master corresponde al coordinador y a QA, no a cada desarrollador individual.

## Controles disponibles y limitaciones reales

El backend tiene develop, qa y master, y master es la rama predeterminada. GitHub rechazó la protección de ramas del repositorio privado por el plan actual (issue #10). Por tanto, el flujo está documentado y validado por CI, pero no está técnicamente protegido contra pushes directos. No se cambió privacidad ni plan. El frontend conserva main como base de su PR; no se afirma que tenga las mismas ramas largas.

ANÁLISIS TÉCNICO / 07

# Trazabilidad de requisitos funcionales

Las fichas funcionales visibles ocupan las páginas físicas 9-18. La siguiente matriz no reemplaza una ejecución completa de aceptación. **Verificado**: caso ejercitado; **Parcial**: existe código o interfaz, faltan validaciones de aceptación; **Pendiente**: no hay evidencia suficiente.

| FRQ | Requisito | Resultado y evidencia |
|---|---|---|
| 0001 | Roles y permisos | Parcial: permisos y middleware; gestión completa de roles pendiente de validar. |
| 0002 | Usuarios | Parcial: módulo seguridad/Usuario; CRUD y asignaciones requieren QA. |
| 0003 | Autenticación | Parcial: bcrypt/JWT y acceso HTTP protegido probado; casos de login completos pendientes. |
| 0004 | Cierre de sesión | Verificado: logout revoca y la reutilización del token recibe 401. |
| 0005 | Clientes | Parcial: catálogo Cliente; conservación de históricos debe probarse. |
| 0006 | Categorías | Parcial: módulo Categoria; CRUD pendiente de aceptación. |
| 0007 | Productos | Parcial: Producto y eliminación lógica; validar referencias históricas. |
| 0008 | Búsqueda de productos | Parcial: consultas y selector; filtros completos requieren QA. |
| 0009 | Creación de factura | Parcial: formulario y registro; ciclo de borrador/confirmación requiere aceptación. |
| 0010 | Detalle de factura | Verificado en registro: detalle válido; edición previa requiere QA UI. |
| 0011 | Disponibilidad | Verificado en confirmación directa: insuficiencia rechazada y locks probados. |
| 0012 | Importes | Parcial: cálculos SQL existentes; validar impuestos, redondeos y reglas autorizadas. |
| 0013 | Cliente en factura | Verificado en caso SQL registrado; validaciones comerciales pendientes. |
| 0014 | Consecutivo | Parcial: actualización transaccional existente; unicidad global y anulación requieren QA. |
| 0015 | Confirmación y emisión | Verificado el registro directo y reintento; aceptación de estados pendiente. |
| 0016 | Consulta de facturas | Parcial: reporte disponible; no se probaron todos los filtros del documento. |
| 0017 | Anulación | Parcial: procedimiento dedicado; reversión de stock requiere integración propia. |
| 0018 | Impresión/reimpresión | Parcial: UI/reportes; smoke test jsPDF/AutoTable, falta aceptación del comprobante. |
| 0019 | Entradas de inventario | Parcial: compras y Kardex; operaciones y persistencia requieren QA. |
| 0020 | Salidas de inventario | Parcial: traslados/movimientos; salidas distintas de venta requieren QA. |

Las dependencias referencian FRQ-0021 y FRQ-0022, pero sus fichas no aparecen en la sección revisada. Deben completarse en el documento de requisitos; no se inventan especificaciones originales.

ANÁLISIS TÉCNICO / 08

# Requisitos no funcionales: operación

| NFR | Evaluación | Evidencia o verificación faltante |
|---|---|---|
| 0001 | Interacción eficiente: parcial | Formularios y modales presentes. Medir pasos, confirmaciones y trabajo continuo con usuarios. |
| 0002 | Tiempo de respuesta: pendiente | Documento exige hasta 3 s en consultas habituales y 8 s en reportes. No se ejecutó benchmark de aceptación. |
| 0003 | Concurrencia: parcial | Se probaron diez transacciones rivales esperando stock. No equivale a diez usuarios completos ni carga sostenida. |
| 0004 | Autenticación segura: parcial | Hash de contraseñas, JWT y cookies. Revisar limitación de intentos, dependencias y configuración final. |
| 0005 | Autorización: implementada | Middleware central consulta permisos activos. Política probada; validar la matriz comercial completa con negocio. |
| 0006 | Sesiones: verificado | Expiración JWT y revocación de logout. Token de prueba reutilizado recibió 401. |
| 0007 | Integridad transaccional: verificado en venta directa | Factura/detalle/Kardex y reversión local; otros movimientos no están acreditados por estas pruebas. |
| 0008 | Duplicados: implementado en venta directa | Registro persistente y hash por intento; reintento devuelve factura anterior. Extender a otras operaciones críticas. |
| 0009 | Disponibilidad: pendiente | No se evaluó servidor productivo, reinicio automático, monitoreo ni recuperación de incidentes. |

## La arquitectura permite estas metas; no las acredita sola

Un pool de conexiones y un servidor central no demuestran por sí mismos el umbral de tiempo ni estabilidad bajo carga. El escenario de locks verifica una propiedad de consistencia: una solicitud rival no puede leer disponibilidad libre mientras otra conserva el bloqueo. Es una prueba intencional de espera, no una medición de capacidad.

Para acreditar NFR-0002/0003 se requiere un ambiente representativo, datos de volumen acordado, diez usuarios concurrentes y métricas por operación. Deben registrarse errores, tiempos máximos, percentiles y consistencia final de stock/consecutivos; no basta con observar una petición rápida.

ANÁLISIS TÉCNICO / 09

# Requisitos no funcionales: calidad

| NFR | Evaluación | Evidencia o acción requerida |
|---|---|---|
| 0010 | Usabilidad: parcial | Estilos y componentes compartidos. Validación con usuarios y dispositivos del negocio pendiente. |
| 0011 | Navegadores: pendiente | Build genera recursos web; falta matriz Chrome/Edge/Firefox y equipos del entorno real. |
| 0012 | Respaldo: pendiente | No hay evidencia de agenda, retención, almacenamiento protegido ni backup completo verificado. |
| 0013 | Restauración: pendiente | Debe restaurarse un respaldo en QA y comprobar factura, detalle, stock y Kardex con la API. |
| 0014 | Auditoría: parcial | Campos de usuario/fecha y módulo de consulta. Integridad de históricos y cobertura total requieren validación; F04 sigue pendiente. |
| 0015 | Mantenibilidad: mejora implementada | Fachada, contratos y adaptador separan facturación. Otros controladores aún reúnen lógica y acceso a datos. |
| 0016 | Validación/errores: parcial | Venta directa valida antes de escribir y controla errores. Otros controladores todavía requieren uniformar respuestas y evitar detalles internos. |
| 0017 | Escalabilidad: alineación estructural | Módulos y cliente/API/datos separados. No se demuestra escalabilidad con carga real ni crecimiento del almacenamiento. |
| 0018 | HTTPS: pendiente de despliegue | Cookies Secure en producción y CORS configurable. No se verificaron proxy, certificado ni redirección HTTP del servidor final. |

## Dictamen de cumplimiento

El núcleo de arquitectura del capítulo II queda alineado en el caso de confirmación directa de factura: cliente-servidor, tres niveles, TypeScript backend y fachada transaccional. El documento completo sigue presentando cumplimiento parcial. Cambiar el patrón no sustituye pruebas de aceptación, infraestructura de operación ni completitud del modelo de datos.

Los modelos UWE de casos de uso, clases, presentación, procesos y navegación (págs. 27-52) constituyen especificaciones de diseño. La presencia de pantallas similares no acredita todas sus condiciones. Para cerrar la aceptación deben vincularse esos escenarios a casos de prueba con resultados y revisión del negocio.

ANÁLISIS TÉCNICO / 10

# Pruebas y evidencia reproducible

| Verificación | Resultado / límite |
|---|---|
| npm test (backend) | 14/14 aprobadas: configuración, SQL parametrizado, validación de factura, coordinación, reintentos, conflicto, rollback y política de permisos. |
| npm run test:integration | 4/4 aprobadas en SQL Server local: registro/reintento/reversión; stock insuficiente; diez rivales bloqueados; HTTP 401/403 y revocación. |
| npm run build (frontend) | TypeScript y Vite aprobados. Bundle principal cercano a 1,49 MB; requiere considerar carga diferida para rendimiento. |
| npm run lint (frontend) | 0 errores, 15 advertencias heredadas. Las advertencias deben atenderse por módulo; no se ocultan. |
| Exportación PDF cliente | Generación en memoria con jsPDF 4.2.1 y AutoTable 5.0.8 verificada. No sustituye revisión visual de cada reporte de negocio. |
| Seguridad de dependencias | Backend: 33 -> 14 avisos (11 altos, 3 moderados). Frontend: 32 -> 1 alto. Sin avisos críticos en el audit final; no equivale a ausencia de vulnerabilidades. |

## Qué se corrigió en dependencias

Se actualizaron paquetes compatibles y lockfiles. Se retiró html2pdf.js, que no se utilizaba, y se actualizó jsPDF/AutoTable con una prueba de generación. La actualización de Puppeteer requirió adaptar la espera de carga del HTML de los reportes. Versiones comprobadas: Express 5.1.0, TypeORM 0.3.31, mssql 11.0.2, Puppeteer 24.43.1, Multer 2.4.0 y TypeScript 5.8.3.

Persisten avisos en cadenas de mssql/tedious, Puppeteer y ts-node-dev; en frontend, xlsx. Algunas correcciones exigen cambio mayor o reemplazo. Debe revisarse su exposición, preparar una migración y probar exportaciones/SQL antes de producción; no se aplicó indiscriminadamente audit fix --force.

## Condiciones de reproducibilidad

Ejecutar npm ci y las migraciones 002-004 en una base local de QA con catálogos y stock. Las pruebas revierten las facturas creadas; SQL Server puede consumir identidades aunque se revierta, por lo que no deben ejecutarse en producción. CI verifica unitarias sin acceso a la base privada. La ejecución local no constituye aprobación independiente de QA.

ANÁLISIS TÉCNICO / 11

# Plan de cierre y despliegue

| Prioridad | Entregable para cerrar la brecha |
|---|---|
| Antes de desplegar | Respaldo y restauración verificados; ejecutar migraciones 002, 003 y 004; publicar backend y frontend con contrato Idempotency-Key compatible. |
| Seguridad | Resolver avisos residuales, usuario SQL de ejecución con privilegios mínimos, TLS real, cookies y CORS de producción. Validar permisos por rol y operación con responsables. |
| Consistencia ampliada | Probar conversión de pedidos, anulación, compras, traslados y devoluciones. Extender idempotencia y coordinación donde el documento las exige. |
| Rendimiento y operación | Acreditar consultas <=3 s, reportes <=8 s y diez usuarios; documentar reinicio, monitoreo y recuperación. Registrar datos/ambiente de la prueba. |
| Documentación | Unificar o explicar Esther/Nathaly; completar FRQ-0021/0022; versionar línea base de esquema. Completar trazabilidad de modelos UWE. |
| Colaboración | Revisión por otra persona, QA y promoción a master. Habilitar protección de ramas cuando el plan lo permita; comprobar reglas efectivas. |
| Evolución gradual | Migrar TS por módulos, resolver advertencias de hooks y dividir bundles. Mantener cada cambio en una feature con criterios de aceptación. |

## Cinco desarrolladores: continuidad del plan existente

F01 ajustes de inventario; F02 devoluciones parciales; F03 notificaciones por usuario; F04 auditoría de catálogos; F05 reposición de inventario. Sus fichas y ramas ya están publicadas. Son trabajo pendiente asignable; este análisis no las marca como implementadas ni reemplaza sus entregables.

## Reversión y mantenimiento

Si falla la nueva versión, revertir la aplicación de forma coordinada y conservar tablas de solicitudes y sesiones para no perder trazabilidad. No eliminar claves ni registros comerciales como mecanismo de rollback. El procedimiento anterior debe recuperarse desde respaldo revisado cuando corresponda. Limpiar sesiones revocadas sólo después de su expiración y mantener sincronizados los relojes de los servicios.

ANÁLISIS TÉCNICO / 12

# Repositorios, fuentes y conclusión

## Repositorios GitHub

Backend: cvargas0503/nathaly-backend

Frontend: cvargas0503/front-nathaly

Cambios publicados en `feature/alineacion-arquitectura-facturacion`. Referencias de código: backend 0fb3b3f (fachada), c593749 (seguridad), a48edb0 (dependencias); frontend d8aad45 (dependencias/TS) y 862966a (facturación tipada). La entrega requiere revisión y QA antes de promover a ramas de entrega.

## Fuentes y evidencia consultadas

[1] Documento base: Sistema Web de Facturación e Inventario para Tiendas Esther, Martínez Guillén y Vargas Dávila, UNI, 16/08/2026. PDF adjunto de 59 páginas; requisitos 9-26, modelos 27-52 y arquitectura 53-59.

[2] Express: Production Best Practices - Security

[3] TypeScript: Migrating from JavaScript

[4] Microsoft: Transactions y Transaction Locking Guide

[5] node-mssql: Transaction y Request

[6] GitHub: About protected branches

[7] Evidencia del proyecto: package-lock.json, controladores/rutas, fachada/adaptador, migraciones 002-004, pruebas unitarias e integración, docs/git-flow.md y docs/requerimientos.

Fuentes técnicas consultadas el 11/10/2026. Los números de versión proceden de los paquetes instalados/lockfiles de esta revisión, no de una recomendación de versiones más recientes. La elección de Facade se fundamenta en el propio capítulo II y en la correspondencia verificada del código.

## Conclusión técnica

El backend aporta un punto central para validar, autorizar y mantener consistente la facturación con el inventario. La fachada reduce el acoplamiento del controlador; la transacción y los bloqueos protegen datos; la idempotencia protege el negocio frente a reintentos. Git Flow y las pruebas hacen cada mejora revisable. Estas decisiones justifican conservar la arquitectura central propuesta y continuar su maduración mediante evidencia, sin introducir microservicios antes de que exista una necesidad medida.